

BraTS 2020 3D segmentation: rebuild learnings

What changed from the original Keras notebook, why each change matters, every technical detail in the new pipeline, what the 10- and 50-epoch runs showed, and what to be ready to explain.

Repo: `D:\brats\brats` · Run analysed: `runs\unet_e10` (50 epochs, resumed from 10; last-epoch weights) ·

Prepared 27 Sep 2026

- | | |
|---|---|
| 1. The short version | 8. Results: 50 epochs, and what changed from 10 |
| 2. Old vs new at a glance | 9. Failure analysis |
| 3. Bugs and weaknesses in the original notebook | 10. Limitations (be upfront about these) |
| 4. New pipeline, file by file | 11. Engineering and environment learnings |
| 5. Data & preprocessing details | 12. General lessons to take away |
| 6. Model, loss and training details | 13. Likely interview questions |
| 7. Evaluation: metrics and conventions | 14. Next steps |
| | 15. Glossary and numbers to remember |

1. The short version

- **Goal shift.** The old project answered "does training run and what is the IoU?". The new one answers "*where* does the model fail, and how was that measured?" That is an evaluation story, which is the angle the Grounding & Evaluation track cares about.
- **Framework.** Keras/TensorFlow notebook → plain Python scripts in PyTorch. TensorFlow ≥ 2.11 has no GPU support on native Windows, so the old run was on CPU: ~29 min per epoch. The new run takes **~1.2 min per epoch** on the RTX 3050 6 GB, roughly 24× faster.
- **Correctness fixes.**
 - Normalization was per axial slice; now it is per volume.
 - The validation set was filtered like the training set; now validation keeps every case.
 - Images and masks were paired by list index; now they are paired by case ID.
 - Metrics were computed in crop space only; now they are computed in the full 240×240×155 space.
 - Accuracy and IoU included the background class; now the report uses BraTS region Dice and HD95.
- **Evaluation added.**
 - WT/TC/ET Dice and HD95, with explicit empty-mask rules.
 - Dice vs tumor volume.
 - Volume agreement in mL (Bland–Altman).
 - How much tumor the fixed crop discards, with the Dice ceiling that implies.
 - Results by grade.
 - A worst-case gallery.
- **Headline result (92 val cases, median Dice, 50 epochs).** WT 0.89, TC 0.89, ET 0.82. At 10 epochs it was 0.82 / 0.72 / 0.66.
- **Main findings.**
 - Small tumors are still the weak regime (ET ≤ 5 mL: about half below Dice 0.5), but size dependence roughly halved with training.
 - Enhancing tumor is predicted in all 6 of the 6 cases that have none, all of them LGG. The false positives are now small, but a volume threshold still can't separate them from true small ET.
 - Mean volume bias is about 0 mL now, but it is still proportional: small tumors are over-called and large ones under-called.

- The worst cases are false positives or crop loss. For cases 181 and 366, the fixed crop is now the binding limit: the model scores 90–93% of the maximum Dice the crop allows.
- Training plateaued around epoch 35. More epochs of the same setup won't help much.

2. Old vs new at a glance

Aspect	Original notebook	New pipeline
Code form	One Jupyter notebook, cells run by hand	Package <code>brats3d/</code> + 3 scripts (<code>prepare_data.py</code> , <code>train.py</code> , <code>evaluate.py</code>) + pytest tests
Framework	TensorFlow 2.17 / Keras, <code>segmentation_models_3D</code>	PyTorch (CUDA 12.8 wheels), loss and metrics written by hand
Hardware	CPU (TF has no native-Windows GPU after 2.10): ~1,600–1,790 s/epoch	RTX 3050 6 GB, fp16 autocast: ~71 s/epoch
Paths	Hardcoded <code>E:\BraTS 2020\...</code> strings with backslashes	<code>--data-root / --work-dir</code> args, <code>pathlib</code> , recursive case discovery
Case 355	Would break or misalign on the Kaggle copy (seg file misnamed)	<code>fix_missing_seg()</code> renames it automatically
Normalization	<code>MinMaxScaler</code> on <code>reshape(-1, 155)</code> , which scales every axial slice independently	Z-score per volume over brain voxels (>0); background stays 0
Channel order	Inconsistent: exploration used [t1ce, t2, flair], the save loop used [flair, t1ce, t2], and plots were mislabeled	Fixed <code>MODALITIES = ("t1ce", "t2", "flair")</code> , stored in the checkpoint and checked at load
Saved data	float64 image (~50 MB) + one-hot float32 mask (~34 MB) per case, named by list index	float16 image (~12.6 MB) + uint8 label mask (~2.1 MB), named by case ID
Split	<code>splitfolders</code> 75/25 on files <i>after</i> filtering; not patient-aware, not stratified	Patient-level, stratified by grade (HGG/LGG), seed 42, written to <code>splits.json</code>
>99% background filter	Applied to all cases, so val was filtered too	Train only (19 train cases dropped); val keeps all 92
Data loading	Generator, no shuffling, pairs files by <code>os.listdir</code> order	<code>Dataset / DataLoader</code> , shuffled, random flips, 2 workers
Normalization layers	None	InstanceNorm after every conv (optional; <code>--norm none</code> = original)
Checkpoints	None saved	<code>last.pt</code> every epoch (resumable: model, optimizer, scaler), <code>best.pt</code> , <code>history.csv/png</code> , <code>config.json</code>
Metrics	Voxel accuracy and IoU (threshold 0.5, all 4 classes incl. background), cropped space	BraTS WT/TC/ET Dice + HD95 in full space, volumes in mL, Bland–Altman, size bins, empty-mask outcomes, crop loss
Reported number	val accuracy 0.955, val IoU 0.460	WT/TC/ET median Dice 0.82/0.72/0.66 (not directly comparable, see §3)
Tests	None	5 pytest tests incl. HD95 vs brute force

3. Bugs and weaknesses in the original notebook

Severity: **HIGH** changes results · **MED** misleading or fragile · **LOW** hygiene.

HIGH Per-slice normalization

`scaler.fit_transform(img.reshape(-1, img.shape[-1]))` reshapes $240 \times 240 \times 155$ into a $57,600 \times 155$ matrix. `MinMaxScaler` scales **each column**, and each column is one axial slice. So every slice was stretched to $[0, 1]$ on its own.

- **Consequence 1.** Intensity is no longer comparable across slices. A slice with only a sliver of brain gets the same range as a slice full of tumor.
- **Consequence 2.** Bright tumor in one slice compresses the rest of that slice.
- **Consequence 3.** MinMax is dominated by a single outlier voxel.

Fix: `zscore_brain()` computes mean and SD over nonzero voxels of the whole 3D volume, per modality. Background (skull-stripped zeros) stays 0.

Learning

Always check which axis a scikit-learn scaler operates on. It scales *features* (columns), and the reshape silently decided that slices were features.

HIGH Validation set was filtered like the training set

The ">99% background in the crop → skip" rule ran *before* the split, so hard cases (tiny or mostly-clipped tumors) never reached validation. The rule keeps 344/369 cases. In the new run the same rule drops 19 train cases, so it would have dropped **6 val cases**.

- From their volumes and crop loss, those 6 are almost certainly 099, 063, 204, 314, 317 and 341.
- Five of these are among the **10 worst** WT cases.
- The old validation therefore removed exactly the regime where the model fails.

Learning

Filters are a training convenience. Applying them to the evaluation set is selection bias. It inflates the reported number and hides the failure mode you most need to see.

HIGH Metrics computed only inside the crop, with background included

- **Crop space.** Tumor outside `[56:184, 56:184, 13:141]` was invisible: the old metric could not penalize it. In the new evaluation, 58/92 val cases lose some WT to the crop, and in 2 cases ~90% of the tumor core is outside.
- **Accuracy.** 0.955 accuracy next to 0.46 IoU is the signature of class imbalance: roughly 98–99% of voxels are background, so predicting "all background" already scores ~0.98.
- **IoU.** The `IOUScore` default averages over all classes including background, and it scores the three raw labels, not the clinically used nested regions WT/TC/ET.

MED Channel order inconsistency

The exploration cells stacked `[t1ce, t2, flair]`, but the save loop stacked `[flair, t1ce, t2]`. The later plotting cell titled channel 0 "t1" and channel 2 "flair", which were wrong for both orders. Training was internally consistent, but any reasoning about "which channel is which" was off. **Fix:** a single `MODALITIES` constant, saved into the checkpoint config. `evaluate.py` refuses to load a checkpoint whose modality list differs.

MED Case identity lost; pairing by list index

- **Pairing by index.** Four separate `sorted(glob(...))` lists (`t2`, `flair`, `t1ce`, `seg`) were zipped by position. If any case lacks one file, for example the Kaggle copy's misnamed case-355 `seg`, every mask after that index shifts by one case. Nothing errors: images silently get another patient's labels.
- **Anonymous file names.** Files were saved as `image_{index}.npy`, so results could not be traced back to a patient, grade, or institution.

- **Loader pairing.** `splitfolders` split the images/ and masks/ folders separately; pairing held only because both had identical counts and the same seed. The generator then paired files by `os.listdir` order.

Fix: everything is keyed by case ID (`BraTS20_Training_XXX`) from discovery to report.

MED No shuffling

The generator walked the file list in the same order every epoch. Batches were identical each epoch, which can bias SGD. **Fix:** `DataLoader(shuffle=True, drop_last=True)`.

LOW Smaller issues

- `val_steps_per_epoch = len(mask_img_list)//batch_size` used the *train* list length (129 steps). By coincidence $129 \times 2 = 258 = 3 \times 86$ val files, so validation ran exactly 3 full passes. The code was wrong, but the effect was harmless.
- **Windows path strings.** Paths like `"E:\BraTS 2020\..."` only work because no backslash formed a valid escape. A folder starting with `t` or `n` would have become a tab or newline. Python 3.12 warns on invalid escapes.
- **Storage.** Saving float64 images and one-hot float32 masks cost ~84 MB per case, versus ~15 MB now. The `splitfolders` copy alone took 16 min.
- `tiffio.imwrite` is deprecated, and the pip install output was left in cells.
- No checkpoints were saved: a kernel crash lost ~4.8 h of training.

Old vs new numbers are not comparable

Val IoU 0.46 and median Dice 0.82 differ in metric (IoU vs Dice), classes (4 labels incl. background vs 3 nested regions), space (crop vs full), validation set (filtered vs unfiltered) and preprocessing. Present the new numbers on their own; do not claim "improved from 0.46 to 0.82".

4. New pipeline, file by file

File	Role
brats3d/config.py	- Shape and crop constants: <code>FULL_SHAPE=(240,240,155)</code>, <code>CROP</code>, <code>CROP_SHAPE=(128,)*3</code>. - Class setup: <code>MODALITIES</code>, <code>NUM_CLASSES=4</code>, <code>REGIONS</code>. - Rules: <code>MIN_FG_FRACTION=0.01</code>, <code>HD95_EMPTY_PENALTY_MM=373.13</code>, <code>SEED=42</code>. <code>WorkPaths</code> defines the preprocessed-folder layout.
brats3d/data.py	Raw I/O with no torch import: - Discovery: <code>find_cases</code> (recursive glob + regex), <code>find_name_mapping</code>, <code>nifti_path</code> (.nii or .nii.gz). - Loading: <code>fix_missing_seg</code>, <code>load_nifti</code> (asserts shape; returns spacing + affine), <code>load_seg</code> (validates labels $\subseteq \{0,1,2,4\}$, remaps 4$\rightarrow$3). - Preprocessing: <code>zscore_brain</code>, <code>load_image</code>. - Evaluation helpers: <code>region_mask</code>, <code>crop_loss</code>, <code>uncrop</code>.
brats3d/metrics.py	<code>outcome</code> , <code>dice</code> , <code>hd95</code> (medpy-style), <code>region_metrics</code> (Dice, HD95, volumes, errors).
brats3d/model.py	<code>ConvBlock</code> and <code>UNet3D</code> (outputs logits).
brats3d/losses.py	<code>soft_dice_loss</code> , <code>focal_loss</code> , <code>DiceFocalLoss</code> (computed in float32).
brats3d/dataset.py	<code>BratsNpy</code> : loads .npy files, applies random flips, yields (x, y, case_id).
brats3d/report.py	<code>summary.md</code> tables, 4 figures, and the worst-case gallery.
prepare_data.py	Parallel preprocessing (4 processes). Writes <code>cases.csv</code> , <code>splits.json</code> and <code>crop_report.md</code> .
train.py	Training loop with AMP, per-epoch checkpoints, resume and history.
evaluate.py	Predicts the crop, pastes it into full size, scores against full GT, and writes the report. Options: <code>--weights last best</code> chooses the checkpoint. <code>--skip-predict</code> re-scores from saved predictions. <code>--save-nifti</code> writes predictions as NIFTI files.
tests/test_metrics.py	Checks that: - identical masks score perfectly; - HD95 matches a brute-force <code>cdist</code> at isotropic and anisotropic spacing; - masks touching the border are handled; - empty-mask conventions hold; - region masks and crop loss are right on a synthetic volume.
Legacy/BraTS_2020.ipynb	The original notebook, moved with <code>git mv</code> so history is kept.

Outputs

D:\data\brats_work\ crop_report.md runs\unet_e10\ runs\unet_e10\eval_last\	images\<case>.npy masks\<case>.npy cases.csv splits.json last.pt best.pt history.csv history.png config.json per_case.csv summary.md dice_hd95_vs_volume.png bland_altman.png volume_scatter.png boxplots.png gallery*.png preds\<case>.npz
---	---

5. Data & preprocessing details

BraTS 2020 basics

- **Cases.** 369 training cases (293 HGG, 76 LGG) from several institutions. Each case has four co-registered, skull-stripped MRI sequences resampled to 1 mm isotropic, 240×240×155: T1, T1ce (post-contrast), T2 and FLAIR.
- **Labels.** 0 background, 1 necrotic / non-enhancing tumor core (NCR/NET), 2 peritumoral edema (ED), 4 enhancing tumor (ET). There is no label 3 in 2020, so it is remapped 4→3 to make the classes contiguous for softmax.
- **Evaluated regions** (nested, which is what the challenge scores):
 - WT (whole tumor) = {1,2,3}
 - TC (tumor core) = {1,3}
 - ET (enhancing tumor) = {3}
- **Clinical meaning.** ET is what enhances with gadolinium. TC is what a surgeon would resect. WT includes edema and is best seen on FLAIR.
- **Validation labels.** The official BraTS validation labels are not public, so this project holds out part of the training set. `name_mapping.csv` gives each case's grade.

Case 355 in the Kaggle copy

The awsaf49 Kaggle mirror ships case 355's label file as `W39_1998.09.19_Segm.nii`. `fix_missing_seg` handles it: for any case with no `_seg` file and **exactly one** unrecognized NIfTI, it renames that file. Zero or several candidates raise an error rather than guessing.

Loading checks

- `load_nifti` reads with `np.asarray(img.dataobj)`, keeping the stored dtype rather than float64 from `get_fdata()`.
- It fails loudly if the shape is not 240×240×155.
- It returns voxel spacing from the header, used for mm and mL, and the affine, used to write NIfTI predictions.
- `load_seg` rounds to uint8 and **asserts labels** $\subseteq \{0,1,2,4\}$ before remapping, so a corrupted file cannot pass silently.

Normalization

```
brain = vol > 0
out[brain] = (vol[brain] - mean(vol[brain])) / max(std(vol[brain]), 1e-8) # background stays 0
```

- **Z-score vs MinMax.** MRI intensities have no absolute unit; scale differs by scanner and protocol. Z-scoring over the brain removes per-scan scale and offset and is robust to single bright outlier voxels.
- **Why only nonzero voxels.** Including the ~85% background zeros would make the mean depend on head size.

Crop and its measured cost

- **The crop.** `[56:184, 56:184, 13:141]` gives 128^3 . It is inherited from the notebook, and $128 = 2^7$ divides cleanly through 4 pooling levels.
- **Measuring it.** `crop_loss(seg)` gives, per region, the fraction of GT voxels outside the crop (NaN if the region is empty). `brain_loss` gives the same for FLAIR>0 brain voxels.
- **Dice ceiling.** If a fraction f of GT is outside the crop, the best possible prediction is all GT inside: $\text{Dice}_{\max} = 2(1-f)G / ((1-f)G + G) = 2(1-f)/(2-f)$. For example, $f = 0.938 \rightarrow 0.117$, and $f = 0.5 \rightarrow 0.667$.

Split

- The split is done on **all 369 cases first**, then the filter applies to training only.
- **Stratified by grade.** Within HGG and within LGG, IDs are sorted (for determinism), shuffled with `np.random.default_rng(42)`, and 25% (rounded) go to val.
- **Val.** 73 HGG + 19 LGG = 92 cases.
- **Train.** 277 cases, of which 258 pass the filter (cropped mask > 1% foreground).
- **Patient level.** Each BraTS case is one patient, so a case-level split is a patient-level split.
- **Institutions.** BraTS 2020 does not release site labels in this copy, so the split can't be made institution-aware.

Storage format

- **Image.** float16, channels-last (128,128,128,3), ~12.6 MB. Z-scores are small numbers, so float16 precision is plenty.
- **Mask.** uint8 labels (128³), ~2.1 MB. One-hot encoding happens on the GPU inside the loss.
- **Parallelism.** `ProcessPoolExecutor(4)`. On Windows, processes are *spawned*, so the script needs the `if __name__ == "__main__":` guard.

6. Model, loss and training details

3D U-Net

Component	Detail
Encoder	5 levels: 16 → 32 → 64 → 128 → 256 filters (<code>base_filters · 2ⁱ</code>). Each level has two 3×3×3 convs (padding 1) + ReLU, with dropout between them.
Dropout per level	0.1, 0.1, 0.2, 0.2, 0.3 (the same as the notebook), mirrored in the decoder.
Down / up	2×2×2 max-pool ×4 (128 → 8 at the bottleneck); 2×2×2 stride-2 <code>ConvTranspose3d</code> up; skip connections concatenated.
Head	1×1×1 conv → 4 logits. Softmax is applied inside the loss and not in the model, which is numerically safer.
Normalization	<code>InstanceNorm3d(affine=True)</code> after every conv, with conv bias disabled since the norm makes it redundant. <code>--norm none</code> restores the original.
Init	<code>kaiming_uniform_(nonlinearity="relu")</code> , bound $\sqrt{(6/fan_in)}$, which equals Keras <code>he_uniform</code> ; biases zero.
Parameters	≈5.6 M (printed at train start).

Why InstanceNorm, not BatchNorm

With batch size 2, BatchNorm statistics are estimated from 2 samples and are very noisy. InstanceNorm normalizes each sample's feature map on its own and is the standard choice in 3D medical segmentation (e.g. nnU-Net). It also absorbs leftover per-scan intensity differences.

Port differences to know (be honest if asked)

1. Keras `Dropout` drops individual activations. This port uses `Dropout3d`, which drops whole feature-map channels.
2. The Keras transposed convs used the default Glorot init; here all convs use He init.
3. InstanceNorm is new.

The architecture is "same topology", not bit-identical.

Loss: Dice + focal (same recipe as the notebook)

```
probs = softmax(logits.float())          # float32 even under autocast
dice_c = (2 * Σ p · y + ε) / (Σ p + Σ y + ε)  # per class, summed over batch AND space (ε = 1e-5)
L_dice = 1 - weighted_mean_c(dice_c)        # weights 0.25 × 4 (equal)
L_focal = mean_voxels Σ_c -y · α · (1-p)γ · log p  # α = 0.25, γ = 2, p clamped to [1e-7, 1-1e-7]
L = L_dice + 1.0 · L_focal
```

- **Dice term.** Optimizes overlap directly and is insensitive to the ~99% background imbalance.
- **Focal term.** Down-weights easy, already-correct voxels by $(1-p)^{\gamma}$, so gradient goes to hard boundary voxels. It also gives smooth per-voxel gradients when Dice is noisy.
- **Batch pooling.** The soft Dice is pooled over the batch (`per_image=False`, like the library default). A tiny tumor paired with a huge one in the same batch contributes little to the loss. That is one plausible contributor to poor small-tumor performance.

Training loop

- **Optimizer.** Adam, lr 1e-4, constant (no scheduler). Run for 10 epochs, then resumed to 50 (same optimizer state, via `--resume`). Batch size 2, 129 steps per epoch.
- **Mixed precision.** `torch.autocast(float16)` for the forward pass; `GradScaler` multiplies the loss before backward so small fp16 gradients don't underflow to zero, then unscales before the step. Mixed precision roughly halves activation memory, which is what lets $128^3 \times \text{batch } 2$ fit in 6 GB.
- **Augmentation.** Independent 50% flips on each of the 3 axes, applied identically to image and mask. The RNG is `torch.rand`, not `numpy`: `DataLoader` seeds torch per worker, whereas `numpy`'s RNG can be duplicated across workers, a known bug that gives every worker identical "random" flips.
- **DataLoader.** `shuffle=True`, `drop_last=True` (so batch size never becomes 1), `pin_memory`, 2 persistent workers. Validation uses batch size 1.
- **Monitoring.** Hard (argmax) region Dice in crop space, pooled per batch. This is only a training monitor. The reported numbers come from `evaluate.py` in full space, per case.
- **Checkpoints.** `last.pt` every epoch holds model, optimizer, `GradScaler` state, epoch, config and best val loss, so `--resume` continues exactly. `best.pt` is saved on the lowest val loss. `history.csv` is truncated to the resume epoch so there are no duplicate rows.
- **Loading.** `torch.load(..., weights_only=False)`: PyTorch ≥ 2.6 defaults to `weights_only=True`. Only disable it for checkpoints you created yourself, since pickle can execute code.
- **Seeds.** `torch.manual_seed(42)` and `np.random.seed(42)`. cuDNN is not forced deterministic, so reruns will differ slightly.

Why evaluate the last epoch, not the best

Validation doubles as the test set. Picking the checkpoint with the lowest val loss and then reporting on that same set is a mild form of test-set selection, which biases the number upward. `--weights last` is the default. In this run val loss fell every epoch, so `best.pt` = `last.pt` = epoch 10 anyway.

7. Evaluation: metrics and conventions

Full-space scoring

The 128^3 prediction is argmaxed, pasted into a zero $240 \times 240 \times 155$ volume (`uncrop`) and compared to the **original, uncropped** GT. Tumor outside the crop therefore becomes false negatives, which is what a real deployment would see.

Dice

$\text{Dice} = 2|P \cap G| / (|P| + |G|)$, computed per case and per region.

- **Scale-free.** A 1 mL error is catastrophic for a 2 mL tumor and negligible for a 150 mL one. That is exactly why Dice correlates with size.
- **Mean vs median.** Report the median [IQR] alongside the mean $\pm$ SD, because the distribution is skewed with a long lower tail.

HD95 (95th-percentile Hausdorff distance)

1. **Surface voxels.** Surface = mask minus its erosion with the 6-connected (face) structuring element.
2. **Directed distances.** For each surface voxel of P, take the Euclidean distance (mm, using voxel spacing) to the nearest surface voxel of G, via `distance_transform_edt(~surface, sampling=spacing)`, and vice versa.
3. **Pool and take the percentile.** Pool both directions and take the 95th percentile. This is the `medpy.metric.hd95` definition.
4. **Speed.** Work inside the bounding box of PUG plus a 2-voxel margin; the result is exact because all surface points lie inside the box.
5. **Border handling.** Pad 1 voxel of zeros so a mask touching the box edge still gets a surface there.

Implementation detail worth knowing

HD95 definitions differ between libraries. Pooling both directions (medpy, used here) is different from taking the max of the two directed 95th percentiles (e.g. MONAI's default). Numbers are close but not identical. Always state which one you used. The test suite checks this implementation against a brute-force `scipy.spatial.distance.cdist` version.

Why HD95 and not max Hausdorff: the max is set by a single stray voxel. The 95th percentile measures boundary error while ignoring the worst 5% of outliers. It complements Dice: Dice counts overlap, HD95 measures how far off the boundary is, and it catches distant false-positive blobs (e.g. case 314, 96 mm).

Empty-mask conventions (explicit, BraTS style)

Outcome	GT	Pred	Dice	HD95	When it happens
both_present	>0	>0	formula	formula	normal case
both_empty	0	0	1	0	correctly says "no ET"
fp_empty_gt	0	>0	0	373.13 mm	hallucination (6 ET cases here)
fn_empty_pred	>0	0	0	373.13 mm	complete miss (0 here)

- **Penalty value.** 373.13 mm = $\sqrt{(240^2 + 240^2 + 155^2)}$, the diagonal of the volume.
- **Why the rules are needed.** Without them, Dice = 0/0 is undefined, and dropping those cases hides the most clinically relevant errors.
- **Both views reported.** The report shows results with the convention and restricted to defined cases, because one 373 mm penalty dominates any HD95 mean.
- **Where it matters.** The ET mean HD95 is 41.0 mm with the penalties and about 12 mm (median 12.0) without them.

Volume agreement

- **Volume.** mL = voxel count $\times$ (sx·sy·sz) / 1000. At 1 mm³, 1,000 voxels = 1 mL.
- **Errors.** Signed error (pred – GT), absolute error, and relative error (GT>0 only; otherwise undefined).
- **Bland–Altman.** Plots the difference against the mean of the two measurements.
 - Bias = mean difference.

- 95% limits of agreement (LoA) = $\text{bias} \pm 1.96 \cdot \text{SD}$.
- A slope in the cloud reveals *proportional* bias.
- **Pearson r measures association, not agreement.** WT volumes correlate at $r = 0.885$, yet the LoA are $[-31, +75]$ mL, over 100 mL wide. A method that always doubled the volume would have $r = 1$ and terrible agreement.

Size dependence

Spearman ρ between GT volume and Dice is rank-based, so it detects any monotone relationship and is robust to the skewed volume distribution. Dice is also reported per volume bin: (0,1], (1,5], (5,20], (20,50], >50 mL, with the % of cases with Dice < 0.5 and the % missed entirely.

8. Results: 50 epochs, and what changed from 10

92 val cases (73 HGG / 19 LGG), unfiltered. Full-space metrics, BraTS empty-mask convention, last-epoch weights (epoch 50, which is also the lowest val loss, so best.pt = last.pt again).

Region	Dice median [IQR]	Dice mean $\pm$ SD	HD95 median [IQR] mm	HD95 mean mm	Vol. bias [95% LoA] mL	r
WT	0.894 [0.826, 0.925]	0.856 $\pm$ 0.116	5.1 [3.2, 10.8]	9.3	+2.2 [−29.1, +33.4]	0.962
TC	0.887 [0.738, 0.932]	0.785 $\pm$ 0.233	4.2 [2.2, 8.3]	9.2	−1.0 [−23.6, +21.7]	0.935
ET	0.819 [0.638, 0.892]	0.697 $\pm$ 0.283	2.2 [1.4, 10.3]	30.7	−1.0 [−11.3, +9.4]	0.958

- **ET on defined cases.** On the 86 cases with ET in GT, ET Dice is 0.746 ± 0.222 (median 0.832). ET HD95 has a median of 2.2 mm and a mean of 6.8 mm. With the 6 penalty cases the mean is 30.7 mm, so the penalties make up about 78% of the reported mean.
- **Context.** Well-trained BraTS models reach roughly 0.90 / 0.85 / 0.80 (WT / TC / ET). This single 3-modality U-Net with a fixed crop is now in that range on medians. Means are lower because of the small-tumor, LGG and crop-clipped tail.
- **Mean vs median.** The TC mean of 0.785 is 0.10 below its median of 0.887, and ET is 0.12 below. A handful of near-zero cases (314, 181, 232, 361) pull the means down. Always report both.

Change from 10 to 50 epochs (same 92 cases, same protocol)

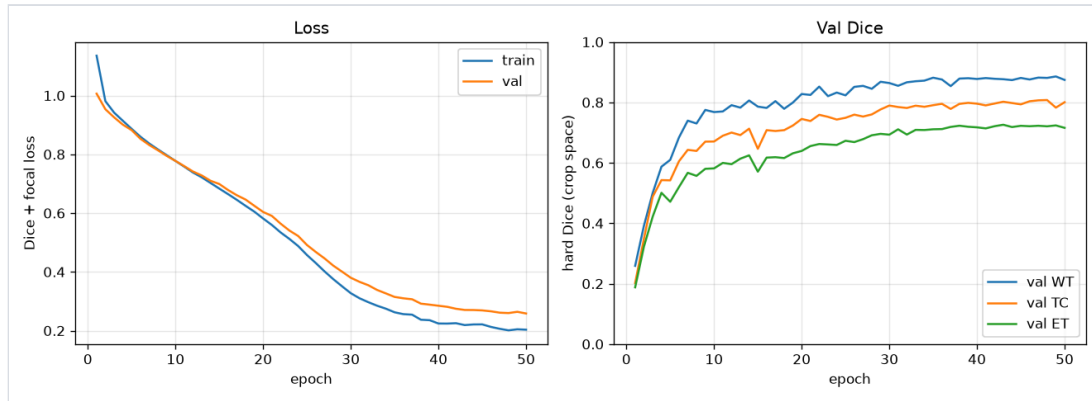
Metric	WT	TC	ET
Median Dice	0.820 $\rightarrow$ 0.894	0.720 $\rightarrow$ 0.887	0.658 $\rightarrow$ 0.819
Mean Dice	0.755 $\rightarrow$ 0.856	0.663 $\rightarrow$ 0.785	0.571 $\rightarrow$ 0.697
Median HD95 (mm)	9.4 $\rightarrow$ 5.1	13.8 $\rightarrow$ 4.2	13.6 $\rightarrow$ 2.2
Volume bias (mL)	+22.0 $\rightarrow$ +2.2	+4.2 $\rightarrow$ −1.0	+5.4 $\rightarrow$ −1.0
LoA width (mL)	106 $\rightarrow$ 62	71 $\rightarrow$ 45	35 $\rightarrow$ 21
Pearson r (volume)	0.885 $\rightarrow$ 0.962	0.851 $\rightarrow$ 0.935	0.872 $\rightarrow$ 0.958
Spearman ρ (Dice vs GT vol.)	0.76 $\rightarrow$ 0.50	0.56 $\rightarrow$ 0.34	0.77 $\rightarrow$ 0.58
ET false positives on ET-absent cases	6/6 $\rightarrow$ 6/6 (but volumes shrank: 297 went from 28 mL to 0.6 mL)		

What this teaches:

- **Some "failure modes" were really under-training.** The +22 mL over-segmentation, the steep size dependence and case 099's 115 mL prediction (now 31 mL) mostly went away with training. If the analysis had stopped at 10 epochs, those would have been reported as properties of the method. *Check that the model has converged before interpreting its failures.*
- **Some failures did not move with training.** ET presence on LGG, crop loss, and LGG TC are problems of the setup or the data, not of the optimization. These are the findings worth talking about.
- **Medians improved more than means.** Most cases got better, but the tail of broken cases stayed broken. That is why tail and worst-case analysis matters.

Convergence

- **Crop-space val Dice.** WT went 0.26 (epoch 1) → 0.77 (10) → 0.86 (30) → 0.87–0.89 (35–50). ET went 0.19 → 0.58 → 0.69 → 0.71–0.72.
- **Plateau.** The model is effectively plateaued from about epoch 35. The loss kept falling, mostly the focal part sharpening confidence, while Dice barely moved.
- **Train–val gap.** WT 0.91 vs 0.87 and ET 0.80 vs 0.72 at epoch 50. That is a mild gap, so the model is not badly overfitting. The gains from here come from the input and the data, not more epochs.
- **Constant LR.** There is no scheduler. A cosine or step decay after the plateau would likely add a point or two.



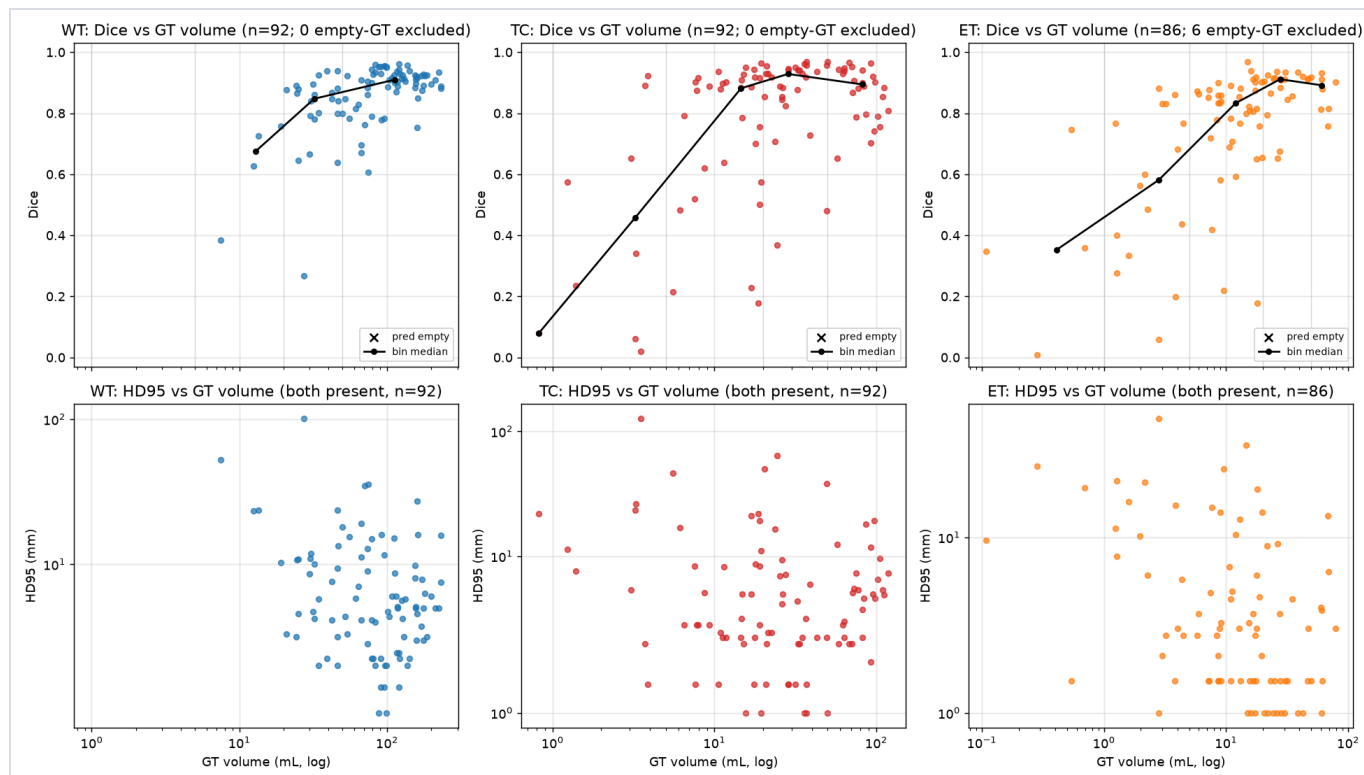
Training history over 50 epochs. Val Dice rises steeply to ~epoch 30 and flattens after ~35; train and val loss track each other.

9. Failure analysis (50 epochs)

F1. Accuracy still depends on tumor size, but less

Region	GT volume bin	n	Dice median	Dice < 0.5	at 10 ep. (median / <0.5)
ET	≤ 1 mL	4	0.35	75%	0.05 / 100%
ET	1–5 mL	16	0.58	44%	0.32 / 75%
ET	5–20 mL	39	0.83	8%	0.66 / 10%
ET	20–50 mL	19	0.91	0%	
ET	> 50 mL	8	0.89	0%	
TC	≤ 5 mL	9	0.08 (≤1 mL), 0.46 (1–5)	5/9	100%
WT	5–20 mL	4	0.68	25%	0.29 / 100%

- **Correlation.** Spearman ρ is 0.50 (WT), 0.34 (TC) and 0.58 (ET), all $p < 10^{-3}$, down from 0.76 / 0.56 / 0.77.
- **No complete misses.** There are still 0 `fn_empty_pred` outcomes. Small lesions are always found, only outlined imprecisely.
- **Why some size dependence always remains.** Dice divides by volume, so a one-voxel boundary error is a large fraction of a 0.3 mL lesion. Even a perfect-boundary-within-1-voxel model shows $p > 0$. Report HD95 alongside Dice for small lesions: ET median HD95 is 2.2 mm, only about 2 voxels.



Dice (top) and HD95 (bottom) against GT volume, log scale. Black line = bin median.

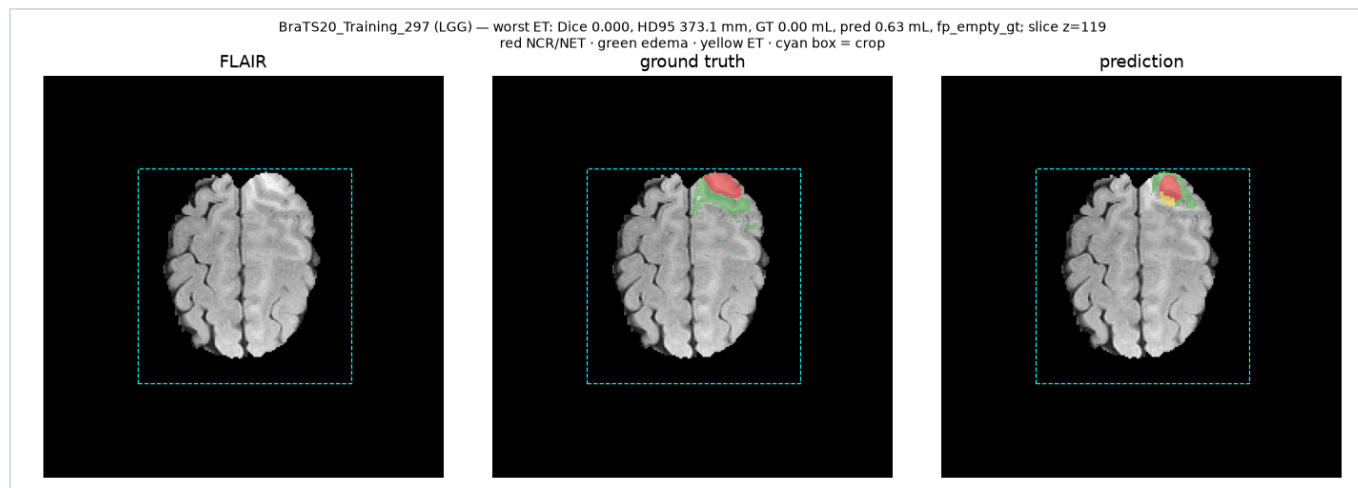
F2. ET is predicted in every case that has none, and a volume threshold can't fix it

- **How often.** 6/6 ET-absent cases get an ET prediction (fp_empty_gt), with 0 both_empty. All six are **LGG**: 262, 297, 305, 306, 319 and 330.
- **Size of the false positives.** They are small now: 0.4, 0.6, 1.2, 4.5, 4.8 and 13.2 mL. At 10 epochs, 297 alone was 28 mL.
- **Cost.** Each costs Dice 0 and HD95 373 mm. Mean ET Dice drops 0.746 $\rightarrow$ 0.697, and mean ET HD95 rises from 6.8 to 30.7 mm.
- **The threshold experiment.** The BraTS-standard trick is "zero ET if predicted ET < k". I swept k on the per-case CSV:

k	FPs fixed (of 6)	True-ET cases wrongly zeroed
0.5 mL	1	3
1 mL	2	4 (incl. 311 at Dice 0.75)
2 mL	3	8
5 mL	5	20

At 1 mL the net change in summed Dice is about +0.8 over 92 cases, which is noise. True small ET (0.1–2 mL, mostly LGG) and spurious ET occupy the *same volume range*. Tuning k on the val set would also be test-set leakage.

- **What would work.** A case-level presence decision: a classification head on the bottleneck, or a rule on predicted core composition such as the ratio of ET to TC, tuned on train folds. This is a calibration problem (is ET present at all?), not a segmentation one.
- **The HGG prior.** The model has learned "tumor core $\Rightarrow$ some enhancement". That is true for almost all HGG, but 6 of the 19 LGG cases have no ET.

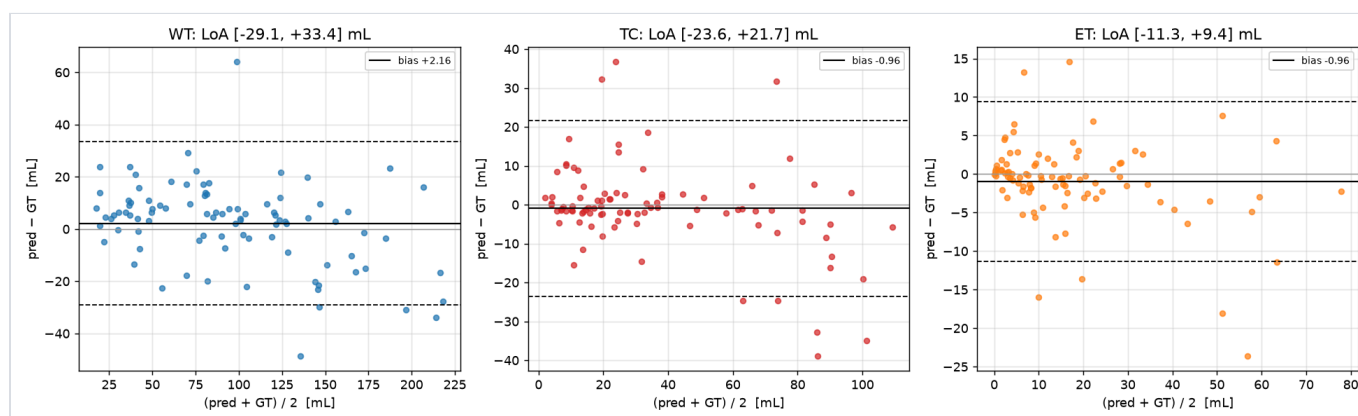


Case 297 (LGG, no ET in GT). The model still predicts a small ET focus (0.6 mL). It scores Dice 0 and HD95 373 mm regardless of how small the focus is.

Metric lesson. Under the BraTS convention, a 0.6 mL false positive costs exactly as much as a 28 mL one. That is a property of the metric, not of clinical harm, which is why the report shows ET both with and without convention-defined cases.

F3. Volume bias is near zero on average, but still proportional

- **Direction.** Bias is +2.2 mL for WT and −1.0 mL for TC and ET, down from +22 / +4 / +5 mL.
- **Agreement.** The limits of agreement narrowed: the WT width went from 106 mL to 62 mL, and Pearson r is 0.96 in every region.
- **Still proportional.** The correlation between volume error and mean volume is about −0.35 in all regions. Below-median WT tumors average +8.6 mL of error; above-median tumors average −4.3 mL. The extremes are +64 and −49 mL.
- **Split direction.** WT is over-segmented in 65% of cases, while TC and ET are over-segmented in only 39–40%. The model paints edema generously but is conservative on core.
- **Relative error.** The median absolute relative error is 12–14% in every region. If volumes were used for response assessment (e.g. RANO), a 12% error is on the order of the change being measured.
- **A near-zero bias can hide errors.** Over-calling small tumors and under-calling large ones cancel out in the mean. Always look at the slope, not just the bias.



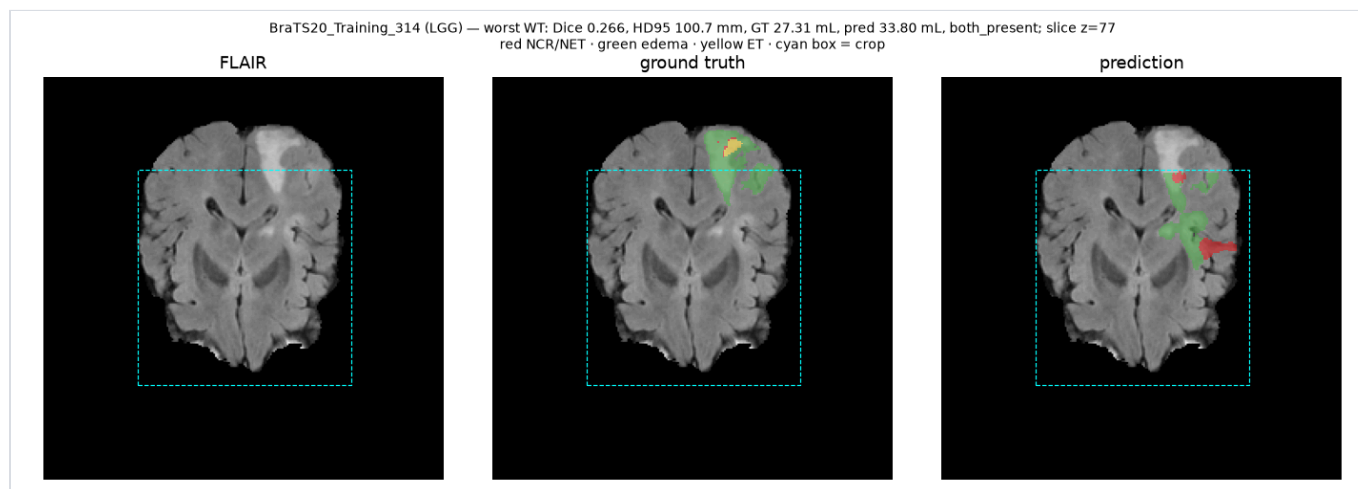
Bland–Altman plots (pred – GT vs mean). Solid = bias, dashed = 95% limits of agreement. There is still a negative slope in every region.

F4. The worst WT cases are false positives or crop loss

Case	Grade	GT mL	Pred mL	WT Dice	WT lost to crop	Driver
314	LGG	27.3	33.8	0.27	61%	crop + distant FP blob
099	HGG	7.5	31.4	0.38	0%	FP (was 115 mL at 10 ep.)

297	LGG	74.9	88.3	0.61	23%	crop + FP
341	HGG	12.5	26.4	0.63	0%	FP (2× over)
366	HGG	46.2	32.8	0.64	44%	crop (under-call)
027	HGG	66.6	130.7	0.67	0%	pure over-segmentation
181	HGG	66.8	44.1	0.69	41%	crop (under-call)

- **Pattern.** In 8 of the worst 10, the prediction is larger than GT. The only two under-calls, 366 and 181, are exactly the cases that lose more than 40% of their tumor to the crop.
- **Old-filter link.** The old training-style validation filter would have dropped 099, 063, 314 and 341. All four are in this worst-10 list.



Case 314 (LGG). GT tumor sits at the frontal edge, 61% outside the cyan crop box; the prediction adds a distant false-positive blob. WT Dice 0.27, HD95 101 mm.

F5. The fixed crop is now the binding limit for several cases

Case	WT lost	TC lost	TC Dice ceiling $2(1-f)/(2-f)$	TC Dice 10 ep.	TC Dice 50 ep.	% of ceiling
314	61%	94%	0.12	0.01	0.02	16%
181	41%	89%	0.20	0.12	0.18	89%
232	27%	64%	0.52	0.18	0.23	44%
297	23%	38%	0.77	0.33	0.37	48%
366	44%	39%	0.76	0.62	0.71	93%

- **How common.** 58/92 cases lose some WT, and 40 of those lose less than 5%. Seven lose $\geq 20\%$: 314, 366, 181, 361, 232, 297 and 221.
- **The key point.** For 181 and 366, the model is already at 89–93% of the best Dice the crop allows. WT tells the same story: 181 scores 0.69 against a ceiling of 0.74, and 366 scores 0.64 against 0.72. *More training cannot help these cases; only a different input window can.* At 10 epochs this cap was partly hidden by other errors. Now it is exposed.
- **Where the tumor sits.** Clipped tumors cluster at the frontal edge of the box.
- **Fix.** Brain-bbox-centred crops, or sliding-window inference over $240 \times 240 \times 155$. The latter needs no retraining: the network is fully convolutional.

F6. Grade: the LGG gap is now mostly ET

Region	HGG Dice (n=73)	LGG Dice	HGG GT mL	LGG GT mL	Explained by size?
--------	-----------------	----------	-----------	-----------	--------------------

ET	0.855	0.419 (n=13)	16.7	2.2	Largely yes: LGG ET is tiny
TC	0.903	0.741 (n=19)	21.5	39.1	No: LGG cores are <i>bigger</i> yet worse
WT	0.896	0.888 (n=19)	87.5	74.9	Equal

- **WT is solved across grades.** The WT gap closed from 0.838 vs 0.809 to 0.896 vs 0.888.
- **TC gap persists (0.16).** LGG core is mostly non-enhancing and looks like edema on FLAIR/T2. T1 is the missing channel that would separate them.
- **ET, two problems.** Small volume in the 13 LGG cases that have ET, and presence errors in the 6 that don't. Together, all LGG ET outcomes are poor.
- **Caveat.** n = 19 LGG, and no confidence intervals were computed.

10. Limitations (be upfront about these)

1. **Random split mixes institutions.** BraTS pools data from ~19 institutions without site labels in this release, so train and val share scanners and protocols. The results are in-distribution and likely optimistic for a new hospital. Real generalization needs site-held-out evaluation.
2. **Validation doubles as the test set.** Mitigated by using last-epoch weights and no tuning, but nothing was evaluated on a truly untouched set.
3. **Fixed crop.** Measured and penalized, not ignored, but it still caps performance on edge tumors.
4. **3 of 4 modalities.** T1 was dropped (inherited). It probably hurts non-enhancing core, as F6 suggests.
5. **Single run, single seed, single split, no CIs.** 50 epochs (plateaued), constant LR. No ensembling, no post-processing, no test-time augmentation. With $n = 92$ and 19 LGG, subgroup comparisons are noisy; bootstrap CIs would be the next step.
6. **Label noise not assessed.** BraTS labels come from multiple raters with known inter-rater variability (especially ET/NCR boundaries). A "failure" could partly be annotation ambiguity. The gallery (e.g. case 099) is where you'd check.
7. **Monitoring metrics differ from reported metrics.** The training history is batch-pooled, crop-space hard Dice, so it will not exactly match the `evaluate.py` numbers.

11. Engineering and environment learnings

Topic	What to remember
TF on Windows	TensorFlow ≥ 2.11 dropped native-Windows GPU support (it needs WSL2). The old notebook ran on CPU without anyone noticing. Always print the device at startup; <code>train.py</code> prints <code>device cuda (RTX 3050...)</code> , <code>amp=True</code> .
PyTorch install	<code>pip install torch --index-url https://download.pytorch.org/whl/cu128</code> . The wheel bundles the CUDA runtime; you only need a new enough NVIDIA driver (yours reports CUDA 13.0, which is backward compatible with 12.8). No separate CUDA toolkit install is needed.
6 GB GPU budget	A $128^3 \times 3$ -channel volume with a 16-base 3D U-Net at batch 2 fits only with fp16 autocast. Fallback: <code>--batch-size 1</code> . Close apps that use GPU memory first.
Windows multiprocessing	Windows <i>spawns</i> child processes, so worker code must be importable and scripts need <code>if __name__ == "__main__":</code> . Otherwise <code>DataLoader</code> workers and <code>ProcessPoolExecutor</code> re-execute the script.
Console encoding	The Windows console defaults to cp1252. Printing "±" or "→" can crash with <code>UnicodeEncodeError</code> . Reports are written with <code>encoding="utf-8"</code> and not printed; set <code>PYTHONIOENCODING=utf-8</code> if needed.
PowerShell 5.1	No <code>&&</code> ; use <code>;</code> or <code>if (\$?) { ... }</code> . Paths with spaces need quotes.
Paths	Use <code>pathlib.Path</code> and CLI arguments, never string-concatenated backslash paths.
Reproducibility	Fixed seed, sorted IDs before shuffling, and the split saved to JSON, so the split is exactly reproducible. Training is <i>not</i> bit-reproducible (cuDNN nondeterminism).
Resumability	Save the optimizer and GradScaler state, not just the weights. Adam's moment estimates matter; resuming without them is effectively a restart with a warm model.
Separate predict and score	Predictions are cached in <code>preds/*.npz</code> , so metric or report changes re-run in seconds with <code>--skip-predict</code> and no GPU.

Test metrics first	Metrics are code too. A brute-force reference implementation on toy masks (including anisotropic spacing and border-touching masks) catches off-by-one and padding bugs that would otherwise contaminate every number.
Data provenance	Kaggle mirrors can differ from the official release (case 355). Validate label values and shapes on load and fail loudly.

12. General lessons to take away

1. **An aggregate score hides the structure.** Median WT Dice 0.82 sounds fine. The stratified view shows the model is useless below ~5 mL, which is exactly where early detection and post-treatment follow-up live.
2. **Evaluate in the space where the output would be used.** Crop-space metrics could never see crop loss.
3. **Never filter the evaluation set with a training heuristic.** Here that single choice would have removed 5 of the 10 worst cases.
4. **Define edge cases explicitly.** Empty masks are not rare corner cases: 6/92 for ET here. Decide the convention up front, and report with and without it.
5. **Correlation $\neq$ agreement.** Use Bland–Altman for measurement comparison. Look for proportional bias, not just the mean bias.
6. **Accuracy is meaningless under extreme class imbalance.**
7. **Look at the pictures.** The gallery turned "low Dice" into specific mechanisms: distant FP blobs, core over-calling, and tumor outside the crop.
8. **Separate confounders before attributing effects.** "LGG is worse" is mostly "LGG ET is small" for ET, but not for TC.
9. **Be precise about what a number means.** Metric definition (HD95 variant), set (filtered or not), space (crop or full), and checkpoint rule (last or best) each move the result.

13. Likely interview questions

Why Dice and HD95 together?

Dice measures volumetric overlap but is insensitive to where the errors are. HD95 measures boundary distance and catches distant false positives. Case 314 has a small volume error (+6.5 mL) but a 101 mm HD95 from a blob in the other hemisphere. They are the official BraTS pair.

Why does Dice depend on tumor size?

Dice normalizes by volume, so a fixed-thickness boundary error of about one voxel is a large fraction of a small object and a tiny fraction of a large one. On top of that, this model also has size-dependent *bias*: it over-calls small tumors.

How did you handle cases with no enhancing tumor?

Explicitly, using the BraTS convention: both empty $\rightarrow$ Dice 1, HD95 0; one empty $\rightarrow$ Dice 0, HD95 373 mm (the volume diagonal). I report both with and without these cases because the penalty dominates means: the ET HD95 mean is 30.7 mm with them and 6.8 mm without. All 6 ET-absent cases were false positives, all of them LGG, and a volume threshold can't separate them from true small ET.

Is 0.89 WT Dice good?

On medians it is near the typical BraTS range (about 0.90 / 0.85 / 0.80) for a single 3-modality U-Net with a fixed crop. But the means are 0.10–0.12 lower for TC and ET, and the gap is a specific tail: small lesions, LGG ET presence, and crop-clipped cases. That tail is the story.

What changed between 10 and 50 epochs?

Medians rose (0.82→0.89 WT, 0.66→0.82 ET), the volume bias went from +22 to +2 mL, and size dependence roughly halved. So several 10-epoch "failure modes" were under-training. What didn't change: 6/6 ET false positives on LGG, crop-capped cases, and the LGG TC gap. Those are structural, and the lesson is to check convergence before interpreting failures.

Would this generalize to a new hospital?

Unknown, and probably worse. The split mixes institutions, so train and val share scanners. A site-held-out split is needed, and BraTS 2020 doesn't expose site labels.

Why last epoch instead of best?

The val set is the reporting set. Selecting a checkpoint on it and then reporting on it is optimistic. Here they coincided anyway.

What's the first thing you'd change?

Replace the fixed crop with sliding-window inference over the full volume. That needs no retraining, and cases 181 and 366 are already at about 90% of their crop-imposed ceiling. Then a case-level ET-presence decision (not a volume threshold, which I showed fails), T1 as a fourth channel, and cross-validation with bootstrap CIs.

What did you get wrong originally?

Per-slice normalization, a filtered validation set, crop-space metrics, accuracy as a headline, index-based file pairing, and training on CPU without realizing it. Owning these is a strength in an evaluation-focused interview.

How do you know your HD95 is right?

It follows medpy's definition, and a pytest compares it against a brute-force all-pairs `cdist` version, including anisotropic spacing and masks touching the border.

What is a Bland–Altman plot telling you here?

At 50 epochs the mean WT bias is only +2 mL, but the negative slope remains: small tumors are over-called by about +9 mL and large ones under-called by about −4 mL, and these cancel in the mean. The 95% limits span about 62 mL despite $r = 0.96$. Correlation measures association, not agreement.

Why not just threshold small ET to fix the false positives?

I swept it. At 1 mL it fixes 2 of the 6 false positives but zeroes 4 real ET cases; at 5 mL it fixes 5 but zeroes 20. The false positives and true small LGG ET have the same size, so it needs a case-level presence signal. Tuning it on val would also leak.

14. Next steps (ordered by expected impact)

1. **Remove the fixed crop.** Sliding-window inference over $240 \times 240 \times 155$ with overlap, or a brain-bbox crop. No retraining is needed, and it lifts the hard ceiling in F5.
2. **Case-level ET presence.** A classifier head or composition rule tuned on train folds (F2 shows a volume threshold fails). Report the effect on both FP and FN.
3. **Add T1** as a fourth channel, aimed at the LGG TC gap. This changes `MODALITIES` and needs re-preprocessing.
4. **LR schedule.** The val metrics plateaued at about epoch 35 with a constant LR. A cosine decay could help.
5. **Uncertainty.** 5-fold CV with bootstrap 95% CIs for medians and subgroup gaps. LGG $n = 19$ is too small for point estimates alone.
6. **External validity.** An external test set (e.g. BraTS 2021 or another cohort) or a site-held-out analysis.

Housekeeping. `runs\unet_e10` now holds the 50-epoch model, because resume overwrote it in place. The 10-epoch numbers survive only in this document and in `history.csv` rows 1–10. Consider renaming the folder to `unet_e50`, and update the README paths if you do.

15. Glossary and numbers to remember

Term	Meaning
WT / TC / ET	Whole tumor {1,2,3} / tumor core {1,3} / enhancing tumor {3} (after 4→3 remap)
NCR/NET, ED	Necrotic / non-enhancing core (label 1); peritumoral edema (label 2)
HGG / LGG	High-grade / low-grade glioma. LGGs often lack enhancement.
T1ce, FLAIR	Post-gadolinium T1 (shows enhancement); fluid-attenuated T2 (shows edema)
HD95	95th percentile of symmetric surface distances, in mm
LoA	Bland–Altman 95% limits of agreement = bias $\pm$ 1.96·SD
AMP / GradScaler	Automatic mixed precision (fp16 forward); loss scaling to avoid fp16 gradient underflow
InstanceNorm	Per-sample, per-channel feature normalization; stable at tiny batch sizes

Number	Value
Cases total / train used / train filtered out / val	369 / 258 / 19 / 92 (73 HGG + 19 LGG)
Volume, crop, voxel	240×240×155 → [56:184, 56:184, 13:141] = 128 ³ ; 1 mm ³ (1,000 voxels = 1 mL)
Filter threshold	> 1% foreground in the crop ($\approx$ 21 mL WT inside the crop)
HD95 empty penalty	373.13 mm
Training	Adam 1e-4, batch 2, 50 epochs (plateau $\sim$ 35), Dice + focal (α 0.25, γ 2), $\sim$ 1.2 min/epoch
Median Dice WT / TC / ET	0.894 / 0.887 / 0.819 (10 ep.: 0.820 / 0.720 / 0.658)
Mean Dice WT / TC / ET	0.856 / 0.785 / 0.697
Median HD95 WT / TC / ET	5.1 / 4.2 / 2.2 mm
Size dependence (Spearman ρ)	0.50 / 0.34 / 0.58 (10 ep.: 0.76 / 0.56 / 0.77)
ET hallucination	6 of 6 ET-absent cases, all LGG; FP 0.4–13.2 mL
WT volume bias [LoA]	+2.2 mL [−29, +33] (10 ep.: +22 [−31, +75])
Crop	58/92 cases clip some WT; 7 clip $\geq$ 20%; worst TC loss 94% (case 314); 181 and 366 at $\sim$ 90% of their ceiling